SMARTBRIDGE | SMARTINTERNZ

Project Documentation

FitBuddy

AI Fitness Plan Generator using Gemini Models

FastAPI • Google Gemini • SQLAlchemy • SQLite • Jinja2

TEAM DETAILS

S.No	Name	Role
1	**Sandhiya M**	**Team Lead**
2	Nivetha N	Member
3	Sathana Shree S	Member
4	Bhavani R	Member
5	Sri Madhumitha A	Member

Team Size: 5 | Team Lead: Sandhiya M

September 2026

Table of Contents

1. Abstract and Project Description **3**
2. Scenarios and Key Features **4**
3. Technical Architecture **5**
4. Prerequisites and Technology Stack **6**
5. Milestone 1 – Model Selection and Architecture **7**
5.2 Application Architecture and Environment Setup 8
6. Milestone 2 – Core Functionalities Development **9**
6.3 Feedback-Based Plan Updating and Storage Layer 10
6.5 Admin View and FastAPI Backend (Activity 2.2) 11
7. Milestone 3 – routes.py Development **12**
8. Milestone 4 – Frontend Development **13**
9. Milestone 5 – Deployment, Testing and Verification **14**
10. Output Screens, Conclusion and Future Scope **15**

Project at a Glance

Item	Details
Project name	FitBuddy – AI Fitness Plan Generator using Gemini Models
Type	Web application with REST API endpoints and an interactive HTML interface
Backend	Python, FastAPI, served by Uvicorn (ASGI)
AI models	Google Gemini 1.5 Pro (plan generation and updates) and Gemini Flash (nutrition tips)
Database	SQLite (fitbuddy.db) accessed through SQLAlchemy ORM
Frontend	HTML, embedded CSS and Jinja2 templates (index, result, all_users)
Core outputs	Personalised 7-day workout plan, nutrition or recovery tip, feedback-updated plan
Team	Sandhiya M (Team Lead), Nivetha N, Sathana Shree S, Bhavani R, Sri Madhumitha A

1. Abstract and Project Description

1.1 Abstract

FitBuddy is a web-based application that uses generative AI to create personalised workout plans and complementary nutrition tips. A user enters basic details such as name, age, weight, fitness goal and preferred workout intensity, and the system returns a structured, day-by-day 7-day routine. The plan can later be refined by submitting natural-language feedback, and every user record and plan is stored so that it can be reviewed at any time.

The application is built on FastAPI and integrates Google's Gemini models: **Gemini 1.5 Pro** produces and revises the workout plans, while **Gemini Flash** generates short, fast nutrition or recovery tips. Data is persisted in SQLite through SQLAlchemy, and the user interface is rendered with Jinja2 templates styled with a gym-themed design.

1.2 Project Description

FitBuddy simplifies the creation of structured and effective fitness routines. The application takes in user information (age, weight, goal and preferred intensity) and generates a customised 7-day fitness plan. Each day contains a warm-up, a main workout with exercises, sets and repetitions, and a cooldown or recovery tip. Users may submit feedback, for example "more focus on cardio" or "include more rest days", and the plan is regenerated by AI to reflect the request.

A nutrition or recovery tip aligned with the selected goal is provided alongside the plan. With a clean user interface and SQLite database support, FitBuddy ensures that each user's data and generated plans are stored and accessible. It exposes both API endpoints (documented automatically at /docs) and an interactive HTML interface for ease of use, plus an admin view for coaches who manage multiple clients.

1.3 Objectives

- Generate structured, goal-specific 7-day workout plans from minimal user input.
- Provide concise, practical nutrition or recovery tips matched to the user's goal.
- Allow users to refine their plan through a feedback loop powered by AI.
- Persist users, original plans and updated plans so both versions are preserved.
- Give admins or coaches a dashboard to review all users and compare plan versions.
- Keep the stack lightweight and API-based, avoiding local GPU hosting or model fine-tuning.

1.4 Problem Statement and Motivation

Generic fitness plans rarely reflect an individual's goal, intensity preference or changing needs, and working with a personal trainer for every adjustment is not always practical. FitBuddy addresses this gap by combining a simple input form with a large language model that already understands structured formatting (days, sections, sets and reps), so a tailored routine and a refinement loop are available on demand.

1.5 Scope

The current release is designed for local deployment with FastAPI and Uvicorn, using SQLite for storage. The architecture is modular so that it can later be prepared for deployment on cloud platforms, as noted in the deployment milestone.

2. Scenarios and Key Features

FitBuddy is organised around four usage scenarios that cover the complete journey from plan creation to administrative oversight.

Scenario	Actor	What happens	Outcome
1. Personalised plan	User	The user enters name, age, weight, fitness goal (e.g. weight loss, muscle gain) and intensity (high, medium or low).	A day-by-day 7-day schedule tailored to the goal and experience level, improving motivation and adherence.
2. Feedback update	User	The user submits feedback on the existing plan, such as “more focus on cardio” or “include more rest days”.	The plan is regenerated by AI and stored as the updated version; the original is preserved.
3. Nutrition / recovery tip	User	A tip is requested based on the selected fitness goal.	A concise suggestion (e.g. include protein in the post-workout meal for muscle gain) that complements the plan.
4. Admin dashboard	Admin / Coach	The admin opens the dashboard to view all registered users and their plans.	Original and updated plans can be compared and user activity monitored, streamlining oversight for trainers.

2.1 Key Features

Feature	Description
AI workout generation	Gemini 1.5 Pro returns a structured plan with warm-up, main workout (exercises, sets, reps) and cooldown for each of the 7 days.
Feedback loop	The original plan and the user’s comments are sent back to Gemini 1.5 Pro, which revises only the relevant parts and keeps the format intact.
Nutrition / recovery tips	Gemini Flash provides fast, friendly, practical tips based on the goal.
Persistent storage	User details, original plan and updated plan are stored in SQLite via SQLAlchemy.
Dual interface	Interactive HTML pages (Jinja2) and JSON API endpoints, with Swagger documentation at /docs.
Admin panel	A tabular view of every user with original and updated plans side by side.
Responsive UI	Flexbox layout, media queries, Roboto typography and a gym-themed background.

2.2 User Roles

Role	Capabilities
User	Fill in the input form, receive a 7-day plan and nutrition tip, submit feedback using the same User ID and receive an updated plan.
Admin / Coach	Open /view-all-users to review registered users, their goals and intensity, and compare original and updated plans.

3. Technical Architecture

FitBuddy follows a modular, layered architecture. The browser talks to a FastAPI backend, which routes requests to templating, workout logic or the admin panel. Workout logic calls the Gemini API, and all results are persisted in SQLite through SQLAlchemy.

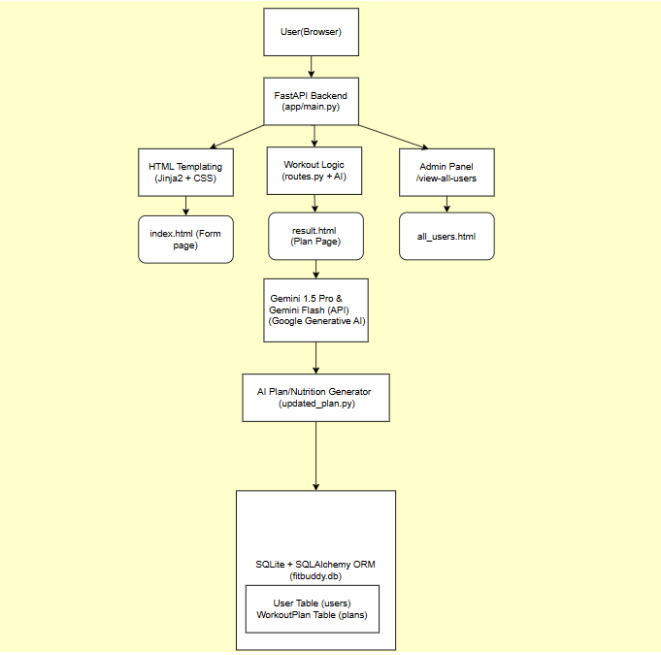


Figure 1: FitBuddy technical architecture

3.1 Request Flow

1. The **user** opens the application in a browser and loads the form page (index.html) rendered by Jinja2.
2. The **FastAPI backend** (app/main.py) receives the request and dispatches it through routes.py.
3. **Workout logic** validates the input and calls the Gemini models: Pro for the plan, Flash for the tip.
4. The **AI generator modules** (gemini_generator.py, gemini_flash_generator.py, updated_plan.py) return the generated text.
5. The user record and plan are saved to **SQLite** (fitbuddy.db) by the SQLAlchemy layer in database.py.
6. The response is rendered into **result.html** (plan, tip and feedback form) or **all_users.html** (admin panel).

3.2 Database Tables

Table	Purpose
users	Stores user id, name, age, weight, goal, intensity and default schedule.
plans (WorkoutPlan)	Stores the user's original AI-generated plan and, after feedback, the updated plan.

4. Prerequisites and Technology Stack

4.1 Prerequisites

No.	Skill / Tool	Reference
1	FastAPI framework knowledge	FastAPI Documentation
2	Gemini API familiarity	Google Generative AI Documentation
3	HTML, CSS and Jinja2 template skills	W3Schools HTML / CSS / Jinja2 tutorials
4	Python programming proficiency	Python Official Docs
5	Version control with Git	Git Documentation
6	SQLAlchemy and SQLite basics	SQLAlchemy Docs
7	Environment setup (pip, virtualenv or conda)	Virtualenv Guide
8	Uvicorn ASGI server	Uvicorn Docs

4.2 Technology Stack

Layer	Technology	Role in FitBuddy
Frontend	HTML, CSS, Jinja2, Google Fonts (Roboto)	Input form, result page, feedback form and admin table
Backend	FastAPI, Pydantic, python-multipart	Routing, form handling and validation
Server	Uvicorn	ASGI server with --reload for development
AI layer	google-generativeai (Gemini 1.5 Pro, Gemini Flash)	Plan generation, plan update and tip generation
Database	SQLite, SQLAlchemy ORM	Persistence of users and plans
Configuration	python-dotenv, .env file	Secure storage of the GOOGLE_API_KEY

4.3 Project Workflow

Milestone	Activities
1. Model selection and architecture	1.1 Select generative AI model; 1.2 Define architecture; 1.3 Set up environment
2. Core functionalities	2.1 Develop core functions; 2.2 Implement FastAPI backend for routing and input processing
3. routes.py development	3.1 Write main application logic in routes.py
4. Frontend development	4.1 Design the user interface; 4.2 Create dynamic Jinja2 templates
5. Deployment	5.1 Prepare for local deployment; 5.2 Test and verify local deployment

5. Milestone 1 – Model Selection and Architecture

This milestone selects the generative AI models from Google's Gemini family and evaluates them against FitBuddy's core functions: generating personalised workout plans, updating them from feedback and providing tailored nutrition or recovery advice.

5.1 Activity 1.1 – Research and Select the Generative AI Model

Project requirements

- Generate structured 7-day workout plans.
- Provide concise and actionable nutrition tips.
- Update workout plans based on user feedback.
- Perform well in real-time, web-based interaction.

Model evaluation

The team first experimented with several Hugging Face models before comparing them with Gemini.

Model	Intended use	Observation
T5 and BART	Text generation (workout and nutrition)	Promising for basic text generation, but limited for structured plans
DistilBERT	Potential intent classification	Not suited to producing long, formatted plans
Bloom	General-purpose generation	Required complex prompting to obtain structured output

These models required hosting with Hugging Face or a local GPU setup, and fine-tuning or complex prompting to achieve structured plans.

Final model selection

Model	Used for	Why
Gemini 1.5 Pro	Structured 7-day workout generation and plan updating	Delivers rich, context-aware, formatted responses reliably
Gemini Flash	Fast, practical nutrition tips	Lightweight and efficient for quick API calls

Reasons for choosing Gemini

- API-based and easy to integrate with FastAPI, with no GPU hosting.
- Naturally understands structured formatting such as days and sections.
- High accuracy with minimal prompt tuning.
- Scalable and fast for feedback-based revisions.

5.2 Application Architecture and Environment Setup

Activity 1.2 – Define the Architecture

Layer	Technology	Responsibility
Frontend	HTML + Jinja2 templates	User input and display: index.html (input form), result.html (plan, tip, feedback), all_users.html (admin panel)
Backend	FastAPI	Receives form data, calls Gemini models, stores user info and plans, renders dynamic templates
AI layer	Google Gemini APIs	Workout generation and feedback updates use Gemini Pro; nutrition tips use Gemini Flash
Database	SQLite + SQLAlchemy	Persistence of users and workout plans

Activity 1.3 – Set Up the Development Environment

Ensure Python and pip are installed, then create and activate a virtual environment, install the libraries and start the server.

```
# create and activate the virtual environment
python -m venv fitbuddy-env
fitbuddy-env\Scripts\activate           # Windows

# install required libraries
pip install fastapi uvicorn jinja2 sqlalchemy python-multipart google-generativeai

# run the server
uvicorn app.main:app --reload
```

Visit http://127.0.0.1:8000 for the application and http://127.0.0.1:8000/docs for API testing.

Project structure

```
fitbuddy/
├── requirements.txt           # Project dependencies
├── app/
│   ├── main.py                # FastAPI entry point
│   ├── routes.py              # Core route handlers
│   ├── database.py            # SQLAlchemy models and DB logic
│   ├── schemas.py             # Pydantic models for validation
│   ├── gemini_generator.py    # Gemini Pro - workout plan generator
│   ├── gemini_flash_generator.py  # Gemini Flash - nutrition tips
│   ├── updated_plan.py        # Feedback-based plan updater
│   ├── nutrition.py           # Nutrition-specific logic (optional)
│   ├── templates/
│   │   ├── index.html         # User input form
│   │   ├── result.html        # Workout plan, tip, feedback
│   │   └── all_users.html     # Admin dashboard
│   └── static/
│       └── images/
│           └── gym-bg.jpg     # Gym-themed background
└── fitbuddy.db                # Local SQLite database (auto-generated)
```

6. Milestone 2 – Core Functionalities Development

6.1 Workout Plan Generation

Function: generate_workout_gemini() | **File:** gemini_generator.py

Implements the core logic that generates a personalised 7-day workout plan from the user's goal (e.g. muscle gain, weight loss) and intensity (low, medium, high). The prompt is sent to Gemini 1.5 Pro, which returns a structured, day-wise plan containing a warm-up (5–10 mins), the main workout (exercise details, sets, reps) and a cooldown or recovery tip. The output is returned to routes.py and rendered on result.html inside <pre> blocks to preserve formatting.

```
def generate_workout_gemini(user_input):
    prompt = f"""
    You are a professional fitness trainer.

    Create a personalized, structured 7-day workout plan for someone with the goal of
    **{user_input['goal']}**, and prefers **{user_input['intensity']}** intensity workouts.

    Each day must include:
    - A warm-up (5-10 mins)
    - Main workout (targeted exercises, sets & reps)
    - Cooldown or recovery tip

    Format:
    Day 1:
    Warm-up: ...
    Main Workout: ...
    Cooldown: ...
    (Repeat for Day 2-7)
    """
    try:
        response = model.generate_content(prompt)
        return response.text
    except Exception as e:
        return f"Error: {e}"
```

6.2 Nutrition Tip Generation

Function: generate_nutrition_tip_with_flash() | **File:** gemini_flash_generator.py

Generates a concise, practical nutrition or recovery tip for the user's goal ("weight loss", "muscle gain" or "general fitness") using Gemini Flash, such as hydration advice, protein suggestions or recovery practices. The tip is displayed beneath the workout plan in result.html.

```
def generate_nutrition_tip_with_flash(goal: str) -> str:
    prompt = (
        f"Give one clear, helpful nutrition or recovery tip for someone focused on '{goal}'. "
        "The tip should be practical, friendly, and easy to understand."
    )
    try:
        response = model.generate_content(prompt)
        return response.text.strip()
    except Exception as e:
        return f"Error generating tip: {str(e)}"
```

6.3 Feedback-Based Plan Updating and Storage Layer

Function: update_workout_plan() | **File:** updated_plan.py

Lets the user revise the original plan by submitting feedback such as “Add yoga” or “Include more cardio”. The original plan and the feedback are passed to Gemini 1.5 Pro, which returns a modified plan. The updated plan is then stored and displayed alongside the nutrition tip.

```
def update_workout_plan(original_plan: str, user_feedback: str) -> str:
    prompt = f"""
    You are a professional fitness trainer assistant.

    Here's the original 7-day workout plan:
    {original_plan}

    User Feedback:
    "{user_feedback}"

    Based on the feedback, revise the relevant parts of the workout plan.
    Keep the format and rest of the plan unchanged if not needed.
    """
    try:
        response = model.generate_content(prompt)
        return response.text.strip()
    except Exception as e:
        return f"Error updating plan: {e}"
```

6.4 User and Plan Storage (database.py)

All user information (ID, name, age, weight, goal, intensity) and AI-generated plans are stored in SQLite via SQLAlchemy ORM. Feedback-based updates are saved in a separate field so both versions (original and updated) are preserved.

Model	Fields	Notes
User (users)	id, name, age, weight, goal, intensity, schedule	schedule defaults to 7 (days)
WorkoutPlan (plans)	user_id, original_plan, updated_plan	Linked to the user through user_id

Function	Purpose
save_user()	Creates a new user or updates the existing record with the same ID
save_plan()	Stores the AI-generated plan as original_plan
update_plan()	Saves the feedback-revised text in updated_plan
get_original_plan() / get_user()	Fetch the stored original plan or the user record

```
def save_user(user_id: int, name: str, age: int, weight: float, goal: str, intensity: str):
    db = SessionLocal()
    existing = db.query(User).filter_by(id=user_id).first()
    if existing:                          # update existing user info
        existing.name = name;  existing.age = age;  existing.weight = weight
        existing.goal = goal;  existing.intensity = intensity
    else:                                 # create a new user
        user = User(id=user_id, name=name, age=age, weight=weight,
                    goal=goal, intensity=intensity, schedule=7)
        db.add(user)
    db.commit()
    db.close()
```

6.5 Admin View and FastAPI Backend (Activity 2.2)

Remaining storage functions

```
def save_plan(user_id: int, plan: str):            # stores the plan in the database
    db = SessionLocal()
    db.add(WorkoutPlan(user_id=user_id, original_plan=plan))
    db.commit();  db.close()

def update_plan(user_id: int, updated_text: str):  # update the plan based on feedback
    db = SessionLocal()
    workout = db.query(WorkoutPlan).filter_by(user_id=user_id).first()
    if workout:
        workout.updated_plan = updated_text
        db.commit()
    db.close()

def get_original_plan(user_id: int):               # fetch the original plan for a user
    db = SessionLocal()
    plan = db.query(WorkoutPlan).filter(WorkoutPlan.user_id == user_id).first()
    return plan.original_plan if plan else None

def get_user(user_id: int):
    db = SessionLocal()
    return db.query(User).filter(User.id == user_id).first()
```

Admin view of all users

Route: /view-all-users | **Template:** all_users.html. The route lists every user with the original and updated plan (showing “N/A” or “Not updated” when a value is missing). The template loops over the records with a Jinja2 {% for user in users %} block and renders them in a structured table.

```
@router.get("/view-all-users", response_class=HTMLResponse)
def view_all_users(request: Request):
    db = SessionLocal()
    users = db.query(User).all()
    user_data = []
    for user in users:
        plan = db.query(WorkoutPlan).filter(WorkoutPlan.user_id == user.id).first()
        user_data.append({"id": user.id, "name": user.name, "age": user.age,
            "weight": user.weight, "goal": user.goal, "intensity": user.intensity,
            "original_plan": plan.original_plan if plan else "N/A",
            "updated_plan": plan.updated_plan if plan and plan.updated_plan else "Not updated"})
    db.close()
    return templates.TemplateResponse("all_users.html", {"request": request, "users": user_data})
```

Activity 2.2 – FastAPI backend: routing and input processing

- **Routing:** all routing logic lives in routes.py; each route is linked to a function from Activity 2.1.
- **Input:** the HTML form in index.html collects username, user_id, age, weight, goal and intensity; FastAPI captures them with Form(...) parameters.
- **Validation:** Pydantic schemas (UserInput, FeedbackRequest) validate the structure of incoming data.
- **Gemini integration:** /generate-workout calls generate_workout_gemini() and generate_nutrition_tip_with_flash(); /submit-feedback retrieves the stored plan and sends it with the feedback to update_workout_plan().

7. Milestone 3 – routes.py Development

routes.py is the operational core of FitBuddy. It bridges the frontend templates, the AI generation modules (Gemini Pro and Flash) and the SQLite database. Each route defines how user input is collected, processed and answered.

7.1 Core Routes

Route	Method	Behaviour
/	GET	Returns the homepage (index.html); no form processing
/generate-workout	POST	Reads Form(...) input, builds a UserInput model, calls Gemini Pro for the plan and Gemini Flash for the tip, saves the user and plan, returns result.html
/submit-feedback	POST	Captures feedback and user_id, retrieves the original plan, calls update_workout_plan(), saves it with update_plan() and renders result.html
/view-all-users	GET	Fetches users and plans and passes them to all_users.html (username, age, weight, goal, intensity, original and updated plans)

7.2 API Endpoint Handlers

```
# 1. API: Generate workout using Gemini Pro
@router.post("/generate-workout/gemini")
async def generate_gemini_workout(request: WorkoutRequest):
    try:
        result = generate_workout_gemini({"goal": request.goal, "intensity": request.intensity})
        return {"model": "gemini-pro", "workout_plan": result}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# 2. API: Generate nutrition tip using Gemini Flash
@router.get("/nutrition-tip")
def get_flash_tip(goal: str):
    tip = generate_nutrition_tip_with_flash(goal)
    return {"goal": goal, "nutrition_tip": tip}

# 3. API: Save user info & generate plan
@router.post("/generate-plan")
def generate_plan(user_data: UserInput):
    try:
        save_user(user_id=user_data.user_id, name=user_data.username, age=user_data.age,
                  weight=user_data.weight, goal=user_data.goal, intensity=user_data.intensity)
        plan = generate_workout_gemini({"goal": user_data.goal, "intensity": user_data.intensity})
        save_plan(user_data.user_id, plan)
        return {"message": "Workout plan generated and saved successfully!", "workout_plan": plan}
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Something went wrong: {str(e)}")

# 4. API: Update workout plan based on user feedback
@router.post("/update-plan/{user_id}", response_model=dict)
def update_user_plan(user_id: int, data: FeedbackRequest):
    original = get_original_plan(user_id)
    if not original:
        return {"error": "Original plan not found for this user."}
    updated = update_workout_plan(original, data.feedback)
    update_plan(user_id, updated)
    return {"updated_plan": updated}
```

All AI responses are shown through result.html, using <pre> blocks to keep the day-by-day formatting readable.

8. Milestone 4 – Frontend Development

The frontend provides a user-friendly, visually structured interface built with HTML, CSS and Jinja2. Each template is rendered dynamically through FastAPI so users can submit input, receive plans, give feedback and, for admins, review data.

8.1 Activity 4.1 – Designing the User Interface

- **Base structure:** index.html is the entry point; the form captures username, user_id, age, weight, goal and intensity, with labelled and semantically grouped fields.
- **Look and feel:** gym-themed background image, Google Fonts (Roboto) and bold typography for a clean, fitness-oriented interface with minimal navigation.
- **Responsive CSS:** embedded styles, Flexbox centring, media queries for mobile, consistent buttons with hover effects, padded inputs with shadows and rounded corners, and a colour scheme readable on a dark background.

Template	Purpose
index.html	User input form for personal details, fitness goal and intensity; submits to /generate-workout
result.html	Displays the 7-day plan, the nutrition tip and a feedback form posting to /submit-feedback
all_users.html	Admin panel listing users with original and updated plans

8.2 Activity 4.2 – Dynamic Templates with Jinja2

```
from fastapi.templating import Jinja2Templates
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
TEMPLATE_DIR = os.path.join(BASE_DIR, "templates")
templates = Jinja2Templates(directory=TEMPLATE_DIR)

return templates.TemplateResponse("result.html", {
    "request": request, "username": username, "user_id": user_id, "age": age,
    "weight": weight, "goal": goal, "intensity": intensity,
    "workout_plan": plan, "nutrition_tip": nutrition_tip })
```

```
{% for user in users %}
<tr>
  <td>{{ user.id }}</td> <td>{{ user.name }}</td> <td>{{ user.age }}</td>
  <td>{{ user.weight }}</td> <td>{{ user.goal }}</td> <td>{{ user.intensity }}</td>
  <td><pre>{{ user.original_plan }}</pre></td>
  <td><pre>{{ user.updated_plan }}</pre></td>
</tr>
{% endfor %}
```

Binding backend data to templates

- **index.html:** form elements use name="..." to bind to backend Form(...) parameters and submit to /generate-workout.
- **result.html:** shows {{ username }}, {{ goal }}, {{ intensity }} and the AI output via {{ workout_plan }} and {{ nutrition_tip }}, plus the feedback form.
- **all_users.html:** loops over all user records as shown above.

9. Milestone 5 – Deployment, Testing and Verification

This milestone deploys FitBuddy on a local system using FastAPI and Uvicorn, ensures all dependencies are installed and verifies the running application. It also prepares the project for future deployment to cloud platforms.

9.1 Activity 5.1 – Preparing for Local Deployment

```
python -m venv venv
source venv/bin/activate          # On Windows: venv\Scripts\activate
pip install -r requirements.txt   # FastAPI, Uvicorn, SQLAlchemy, Google Generative AI SDK
```

Sensitive data such as the Gemini API key is kept in environment variables, either set in the terminal or stored in a .env file loaded with python-dotenv:

```
GOOGLE_API_KEY=your_gemini_api_key_here
```

Keep the .env file out of version control so the API key is never committed.

9.2 Activity 5.2 – Running and Accessing the Application

Start the development server with **uvicorn app.main:app --reload** from the project root (or **uvicorn main:app --reload** when run from inside the app folder). Open http://127.0.0.1:8000 for the user interface; the interactive API documentation is available at http://127.0.0.1:8000/docs, where every endpoint can be explored and tested.

9.3 Test Cases and Results

ID	Test case	Steps / input	Expected result	Status
TC-01	Home page loads	Open http://127.0.0.1:8000	Form with name, user ID, age, weight, goal and intensity is shown	Passed
TC-02	Generate plan	Submit form (e.g. High intensity, fat loss and muscle gain)	Summary of user details and a 7-day plan with warm-up, main workout and cooldown	Passed
TC-03	Nutrition tip	Submit the form in TC-02	Goal-based tip appears below the plan	Passed
TC-04	Feedback update	Enter the same User ID and feedback text	Confirmation message and revised plan are saved	Passed
TC-05	Admin dashboard	Open /view-all-users	Table shows users with original and updated plans	Passed
TC-06	API docs	Open /docs	Swagger UI lists all endpoints	Passed

Screenshots supporting these results are shown in Section 10.

10. Output Screens, Conclusion and Future Scope

10.1 Output Screens

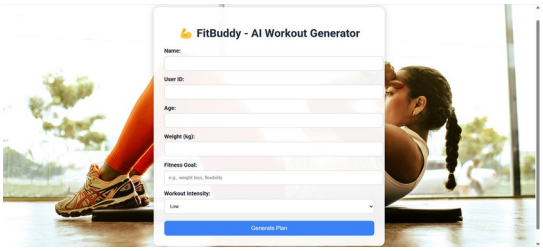


Figure 2: Home page (index.html)

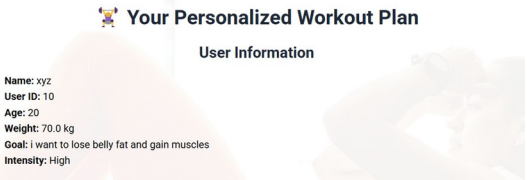


Figure 3: Personalised plan - user information

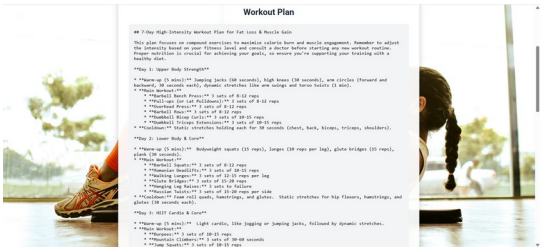


Figure 4: 7-day workout plan



Figure 5: Nutrition tip (Gemini Flash)

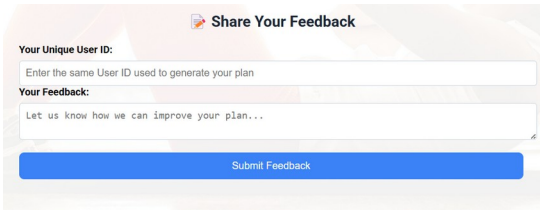
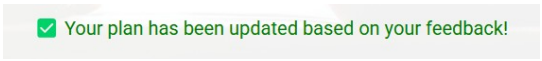


Figure 6: Feedback page



Update confirmation

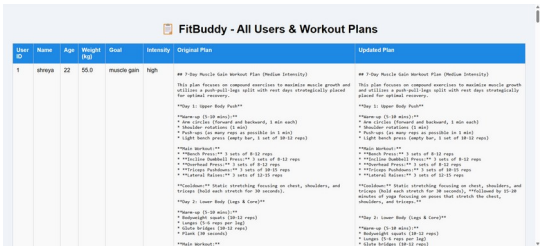


Figure 7: Admin view of all users

10.2 Conclusion

FitBuddy demonstrates how generative AI can be integrated into an everyday wellness workflow. Using FastAPI, Gemini 1.5 Pro and Gemini Flash, SQLAlchemy with SQLite, and Jinja2 templating, it turns a few user inputs into a personalised 7-day plan and a goal-aligned tip, keeps both original and updated plans, and gives coaches a clear overview of all users. The feedback loop keeps each plan adaptive to the user's progress and preferences.

10.3 Future Scope

- Deploy to a cloud platform and move from SQLite to a managed database.
- Add user authentication and role-based access for the admin dashboard.
- Add progress tracking and a version history of plan updates.

Project Team: Sandhiya M (Team Lead), Nivetha N, Sathana Shree S, Bhavani R, Sri Madhumitha A.